IT Workshop II (Cloud Computing): Python for Cloud, Cloud Application Development in Python

September 20, 2026

Python Web Application Framework - Django

- Django is an open source web application framework for developing web applications in Python.
- A web application framework in general is a collection of solutions, packages and best practices that allows development of web applications and dynamic websites.
- Django is based on the Model-Template-View architecture and provides a separation of the data model from the business rules and the user interface.
- Django provides a unified API to a database backend.
- Thus web applications built with Django can work with different databases without requiring any code changes.

Python Web Application Framework - Django

- With this fiexibility in web application design combined with the powerful capabilities of the Python language and the Python ecosystem, Django is best suited for cloud applications.
- Django consists of an object-relational mapper, a web templating system and a regular-expression-based URL dispatcher.

Django Architecture

- Django is Model-Template-View (MTV) framework.
- **Model:** The model acts as a definition of some stored data and handles the interactions with the database. In a web application, the data can be stored in a relational database, non-relational database, an XML file, etc. A Django model is a Python class that outlines the variables and methods for a particular type of data.
- **Template:** In a typical Django web application, the template is simply an HTML page with a few extra placeholders. Django's template language can be used to create various forms of text files (XML, email, CSS, Javascript, CSV, etc.)
- **View:** The view ties the model to the template. The view is where you write the code that actually generates the web pages. View determines what data is to be displayed, retrieves the data from the database and passes the data to the template.

Starting Development with Django

- **Create a Django Project:** django-admin startproject project_name
- **Create a Django App:** python manage.py startapp app_name
- **Define Models in models.py:**
  class Product(models.Model):
  name = models.CharField(max_length=255)
  price = models.DecimalField(max_digits=10, decimal_places=2)
- **Migrate the Database:** python3 manage.py makemigrations
  python3 manage.py migrate
- **Create Views in views.py:**
  from django.shortcuts import render
  from .models import Product
  def product_list(request):
  products = Product.objects.all()
  return render(request, 'product_list.html', 'products': products)

Starting Development with Django

- **Map URLs to Views in urls.py:**
  from django.urls import path
  from . import views
  urlpatterns = [path('products/', views.product_list, name='product_list'),]
- **Create Templates:** Inside the project folder, we create a templates folder. Inside the templates folder, we create a HTML file.
- **Run the Server:**
  python3 manage.py runserver
- Django also offers features like testing, custom middleware, third-party package integration, REST API development (with Django REST Framework) etc.

Design Methodology for IaaS Service Model

Component Design

- Indentify the building blocks of the application and to be performed by each block
- Group the building blocks based on the functions performed and type of cloud resources required and identify the application components based on the groupings
- Identify the inputs and outputs of each component
- List the interfaces that each component will expose
- Evaluate the implementation alternatives for each component (design patterns such as MVC, etc.)

Architecture Design

- Define the interactions between the application components
- Guidelines for loosely coupled and stateless designs - use messaging queues (for asynchronous communication), functional interfaces (such as REST for loose coupling) and external status database (for stateless design)

Deployment Design

- Map the application components to specific cloud resources (such as web servers, application servers, database servers, etc.)

Design Methodology for PaaS Service Model

- For applications that use the Platform-as-a-service (PaaS) cloud service model, the architecture and deployment design steps are not required since the platform takes care of the architecture and deployment.
- **Component Design:** In the component design step, the developers have to take into consideration the platform specific features.
- **Platform Specific Software:** Different PaaS offerings such as Google App Engine, Windows Azure Web Sites, etc., provide platform specific software development kits (SDKs) for developing cloud applications.

Design Methodology for PaaS Service Model

- **Sandbox Environments:** Applications designed for specific PaaS offerings run in sandbox environments and are allowed to perform only those actions that do not interfere with the performance of other applications.
- **Deployment & Scaling:** The deployment and scaling is handled by the platform while the developers focus on the application development using the platform-specific SDKs.

Image Processing Application - Component Design

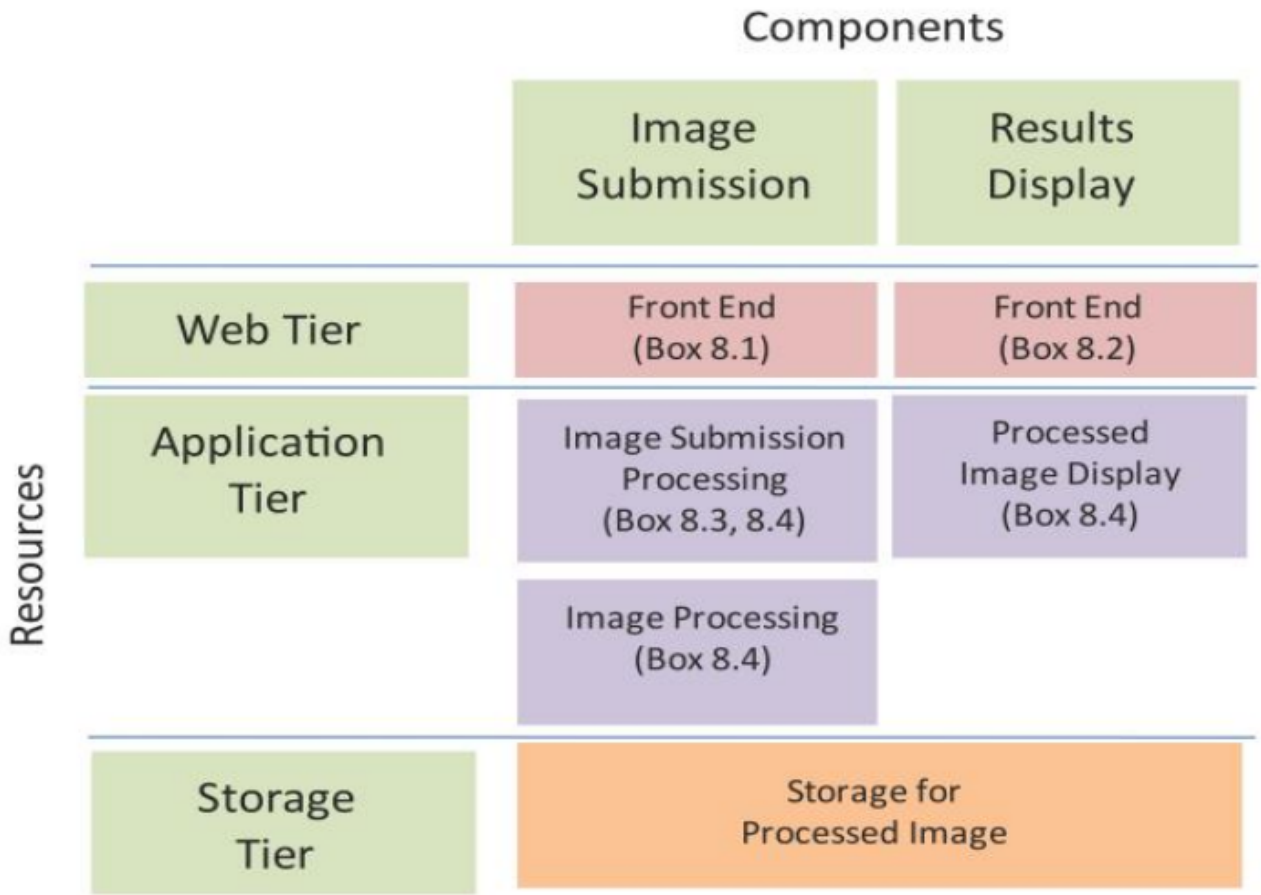


Image Processing Application - Architecture Design

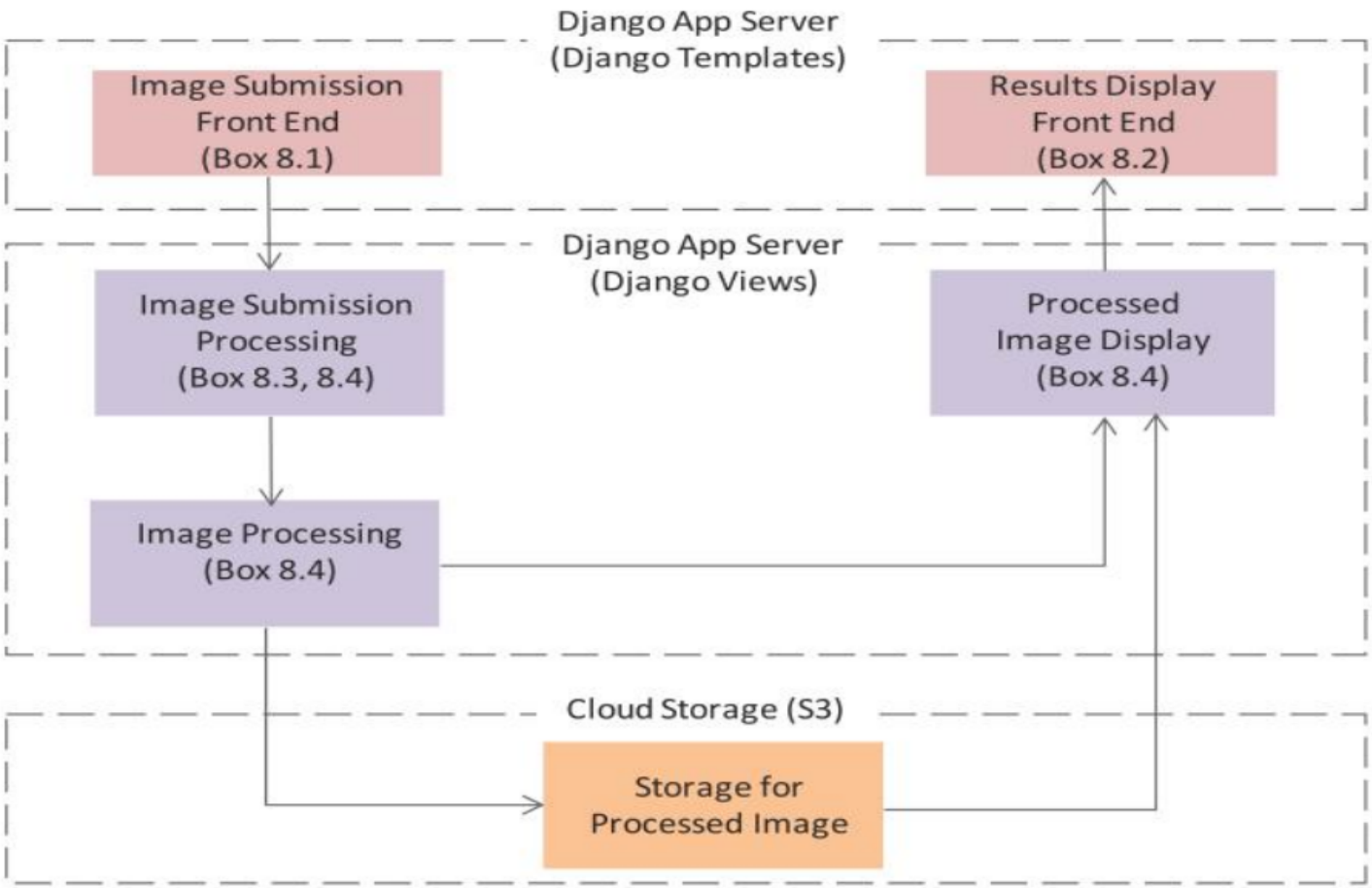
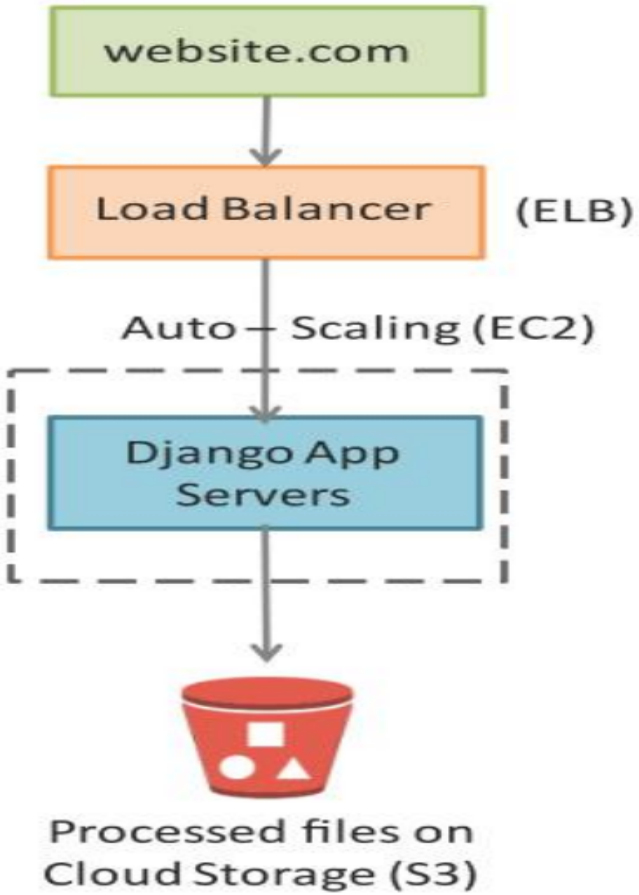
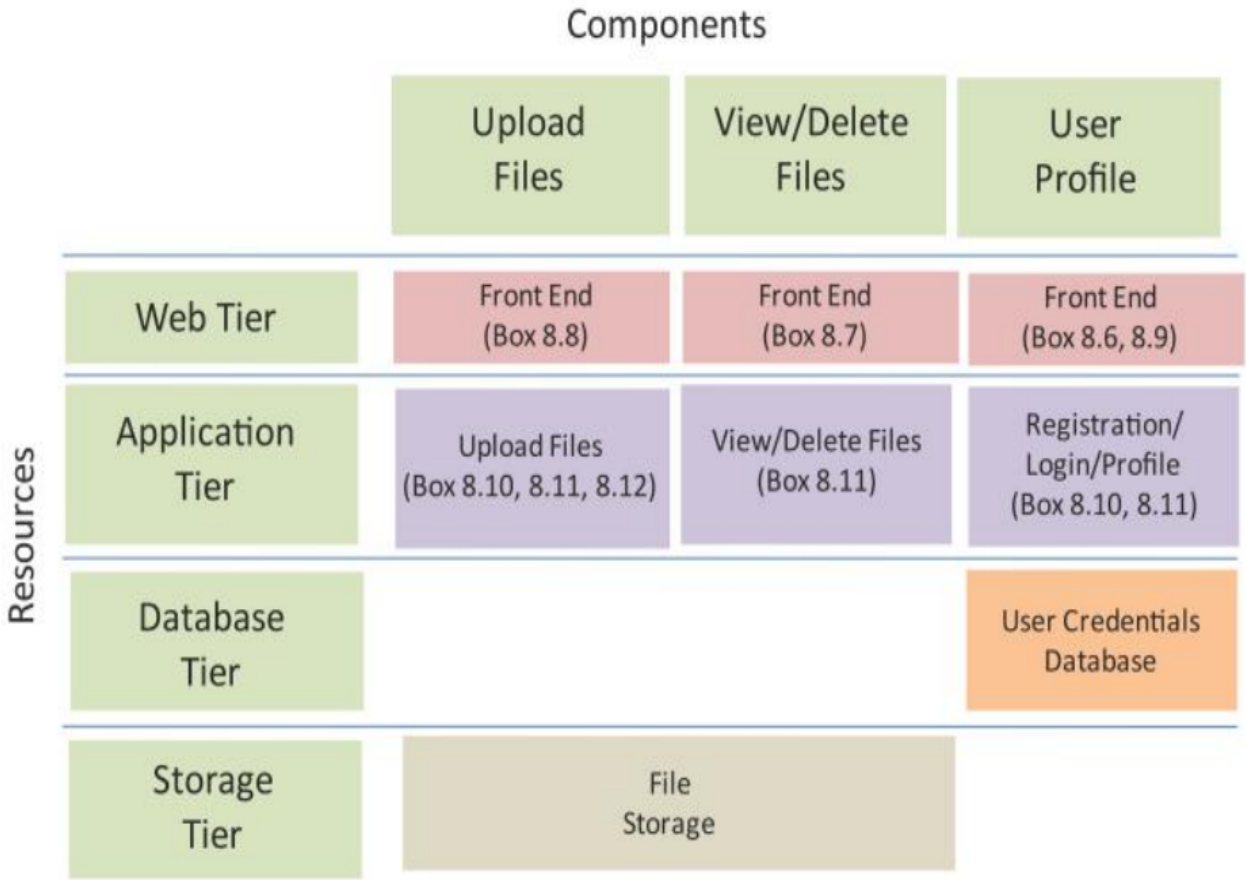


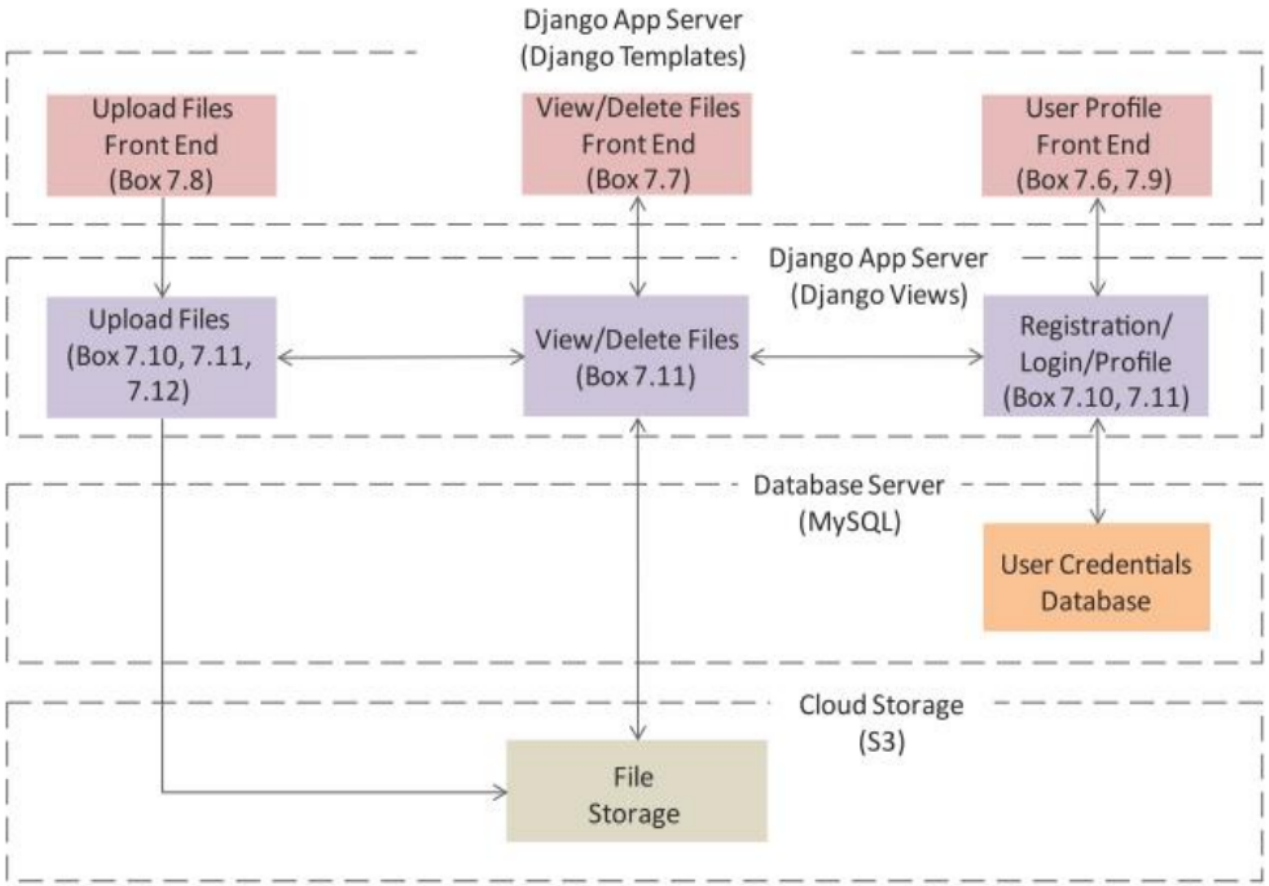
Image Processing Application - Deployment Design



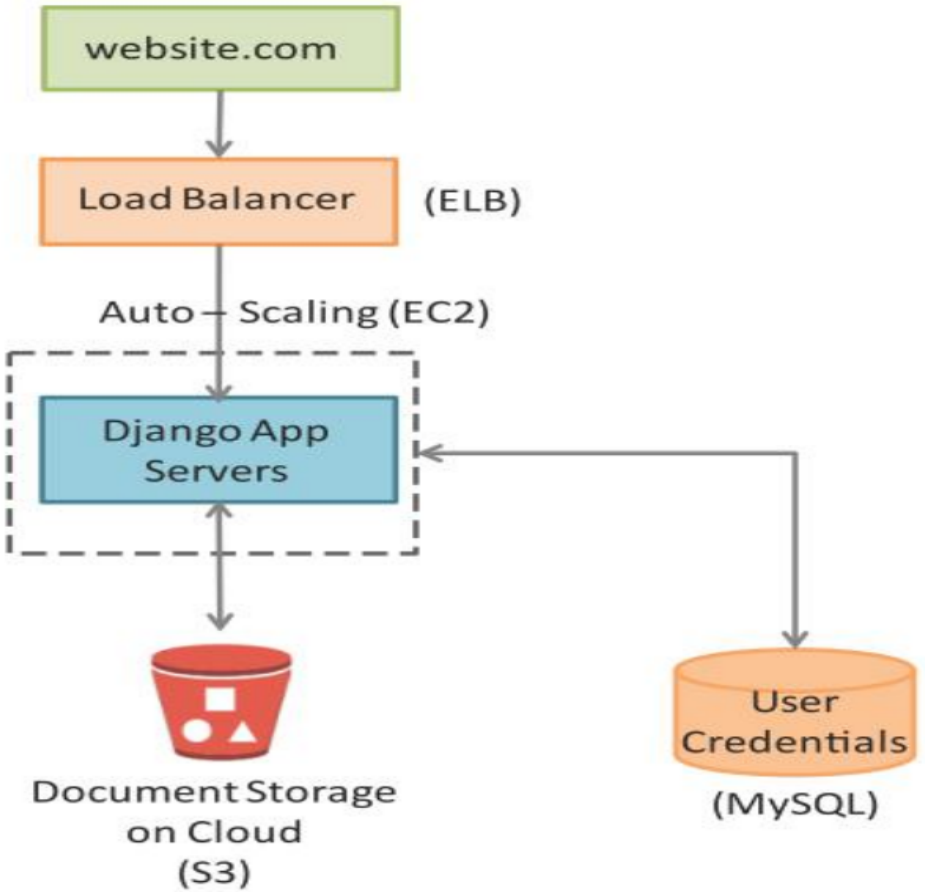
Document Storage Application - Component Design



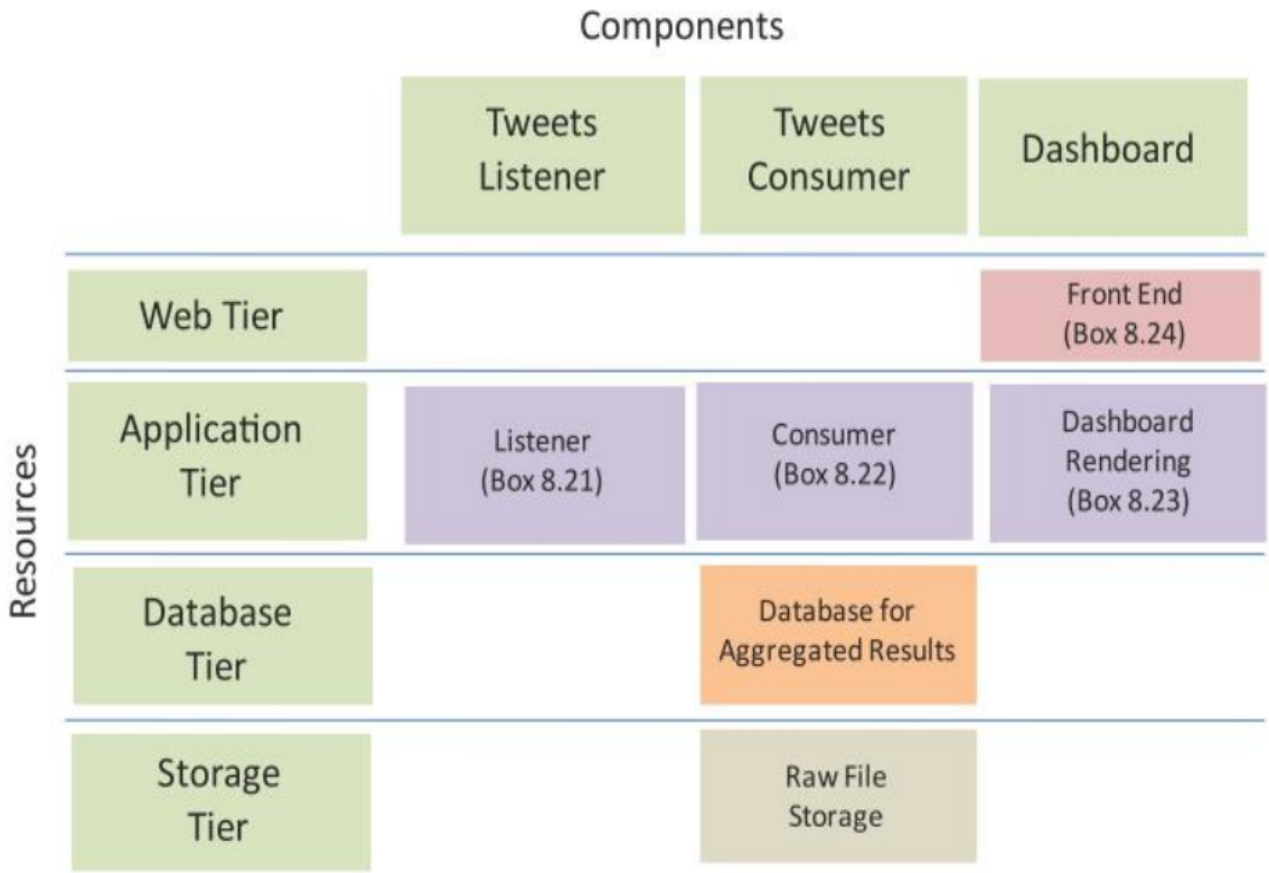
Document Storage Application - Architecture Design



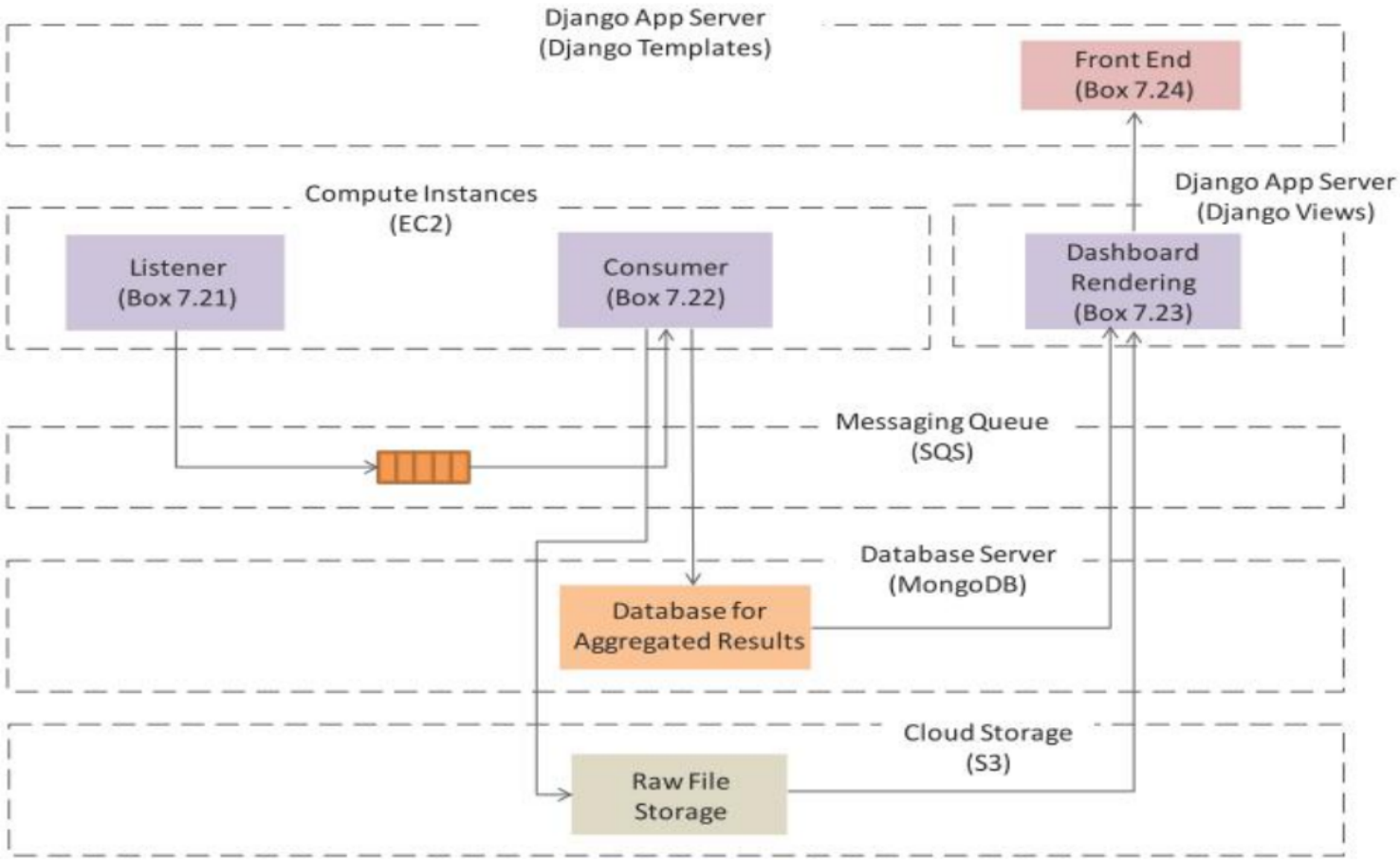
Document Storage Application - Deployment Design



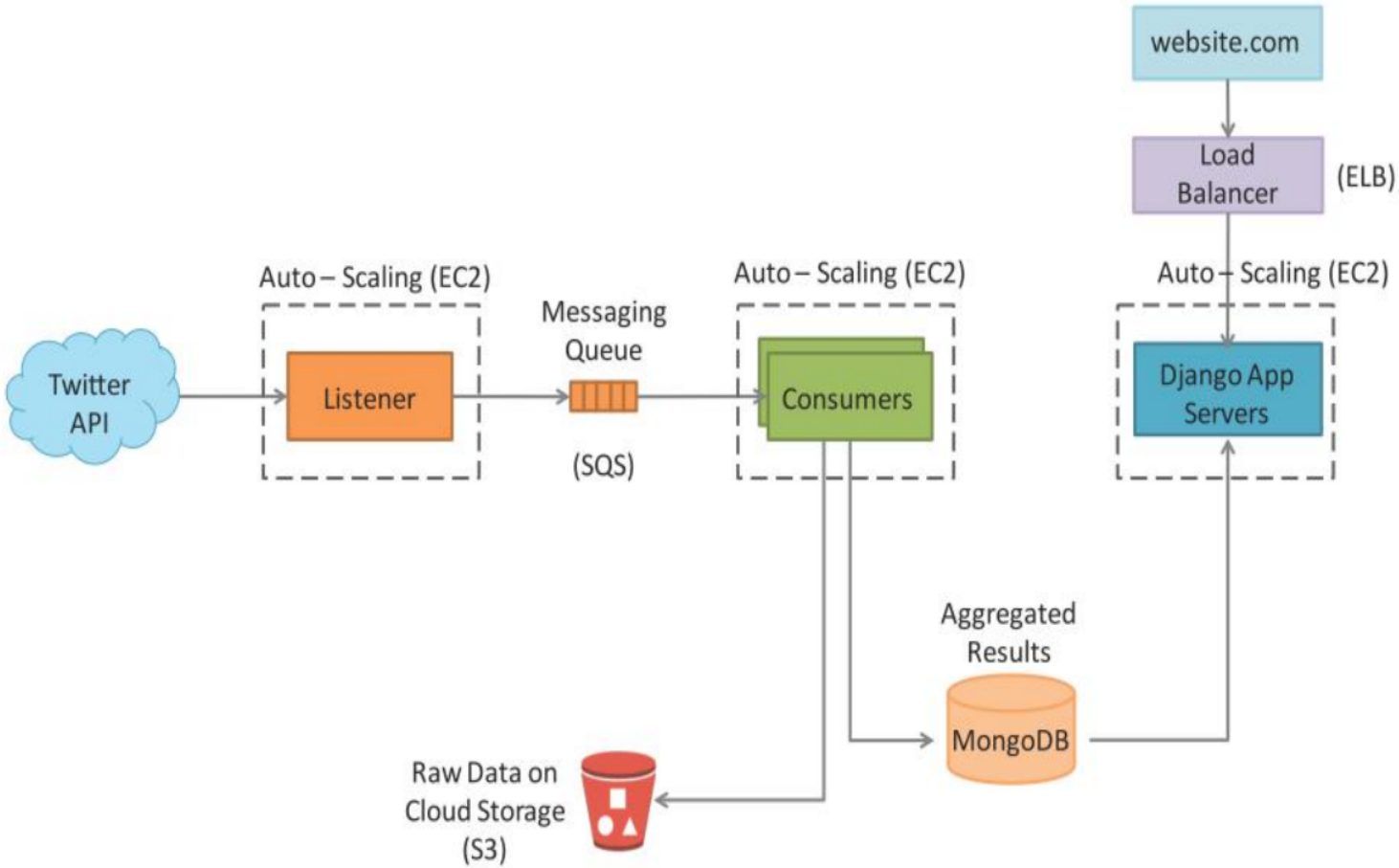
Social Media Analytics Application - Component Design



Social Media Analytics Application - Architecture Design



Social Media Analytics Application - Deployment Design



The End